

Why Vector Search Wasn't Enough for Deal Intelligence

A practical article on combining RAG, Hindsight memory, recall, reflection, risk analysis, and what-if simulation for sales intelligence.

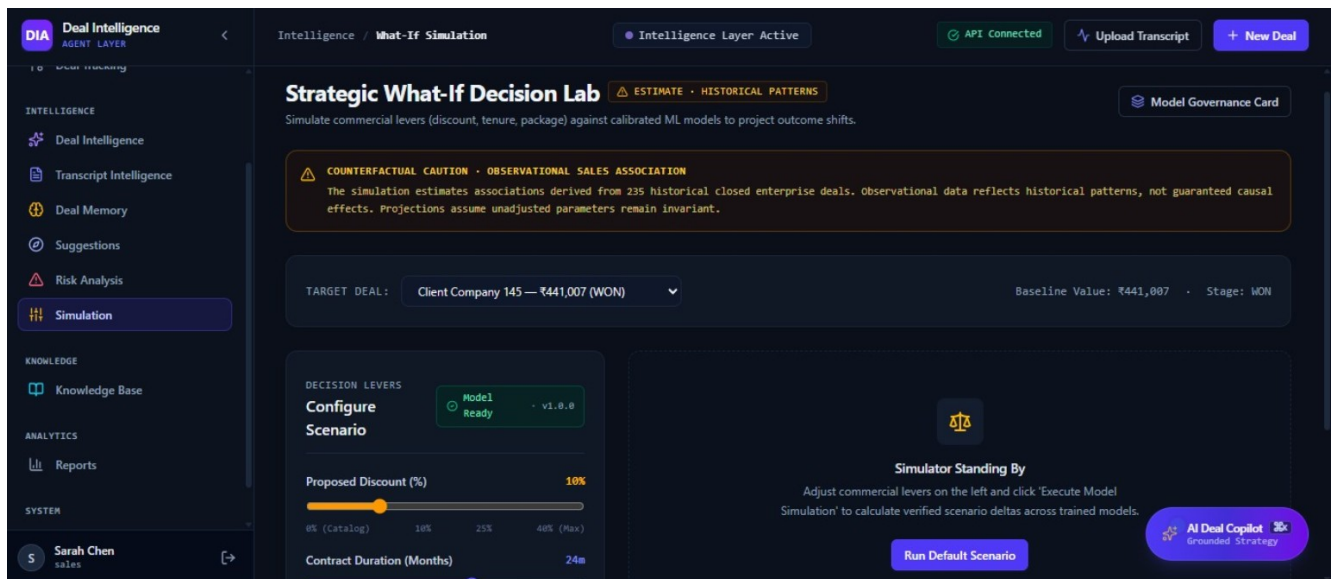


Figure 1. Deal Intelligence interface and intelligence workflow.

The first version of the deal assistant could answer factual questions, but it struggled with the more useful question: “What happened in similar deals, and what did we learn?”

1. What I Built

The Deal Intelligence Agent sits on top of a company's sales data. It does not replace the CRM; it reads existing deal records, call and meeting transcripts, email threads, playbooks, and pricing documents.

The practical questions are: What is this customer really worried about? Have we seen this before, and how did it end? What should I do next, and why? What happens if I offer a discount versus a redesigned package?

The stack is deliberately straightforward: a React/TypeScript front end, a Node.js/Express gateway, a FastAPI service running a LangGraph agent, PostgreSQL for structured data, pgvector for embeddings, and a hosted language model for reasoning.

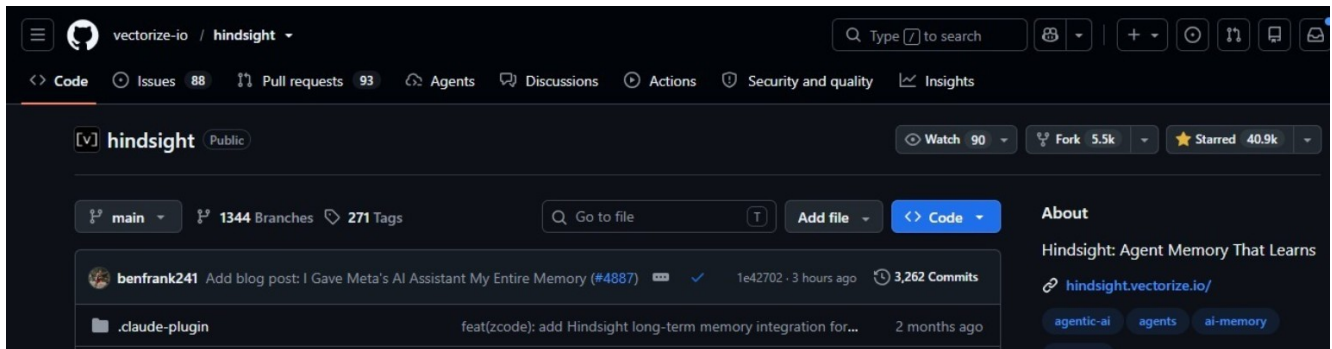


Figure 2. Hindsight GitHub repository used as the memory-layer reference.

2. Two Kinds of “What Do We Know?”

Retrieval-augmented generation is good at answering: “What do our documents say?” A vector index can retrieve relevant text from pricing guides, playbooks, and transcripts. But sales teams also need another kind of knowledge: what happened in completed deals and why.

Two deals can contain nearly identical objection language and still have opposite outcomes. Cosine similarity can identify textual neighbors, but it does not by itself explain which outcome mattered or why.

Knowledge (RAG)	Experience (Hindsight)
Question: What does our material say?	Question: What happened last time, and why?
Unit: Document chunk	Unit: Deal outcome + lesson
Written on upload	Written on deal closure
Failure mode: stale document	Failure mode: wrong analogy

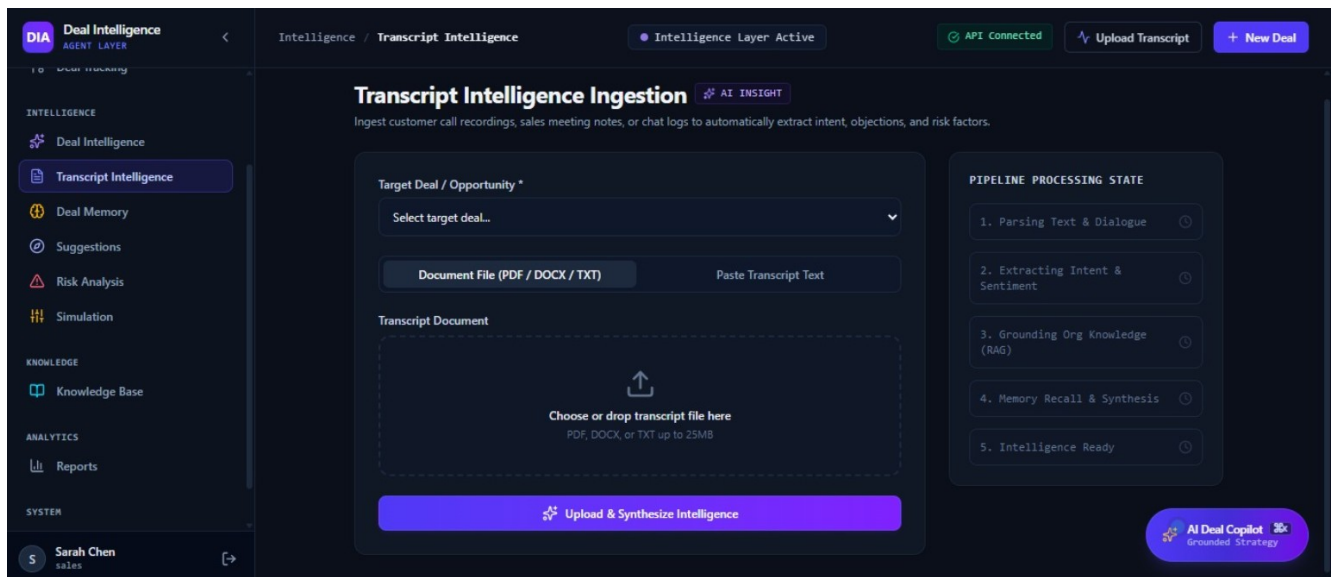


Figure 3. Institutional deal memory: recall → reflect → retain.

3. Retain the Lesson, Not the Transcript

The first design instinct was to push every transcript into memory and let recall sort it out. That created a large, noisy bag of text — effectively RAG with a different name.

Instead, retention happens when a deal is closed. The system records a distilled outcome containing the objection, strategy, approach, reason for the outcome, competitor, and stage where the deal was lost.

A short lesson such as “Pricing objection at negotiation stage; redesigned the package instead of discounting; won” is more useful for future recall than a complete transcript dump.

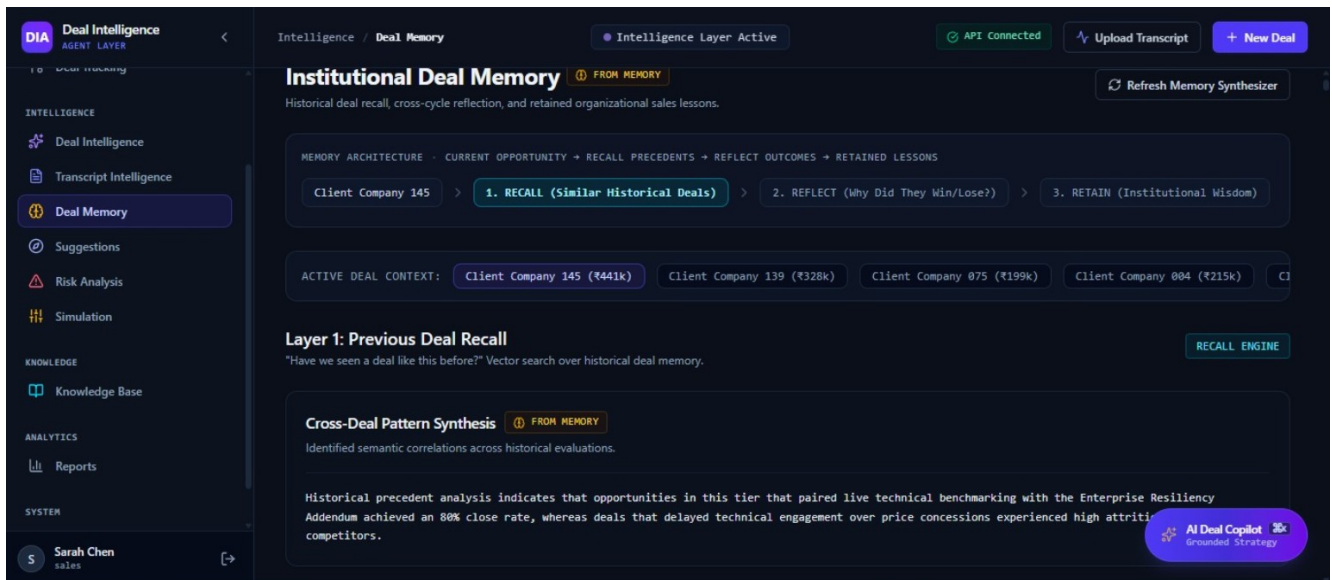


Figure 4. Transcript Intelligence ingestion for extracting intent, objections, and risk factors.

4. Recall: Similarity Is More Than Embeddings

A deal is similar to another when the situation matches: industry, deal size, product, competitors, objections, and requirements — not merely when the words overlap.

Recall therefore combines structured filters with semantic matching. Structured information narrows candidates, while semantic matching over objections and requirements ranks the relevant historical lessons.

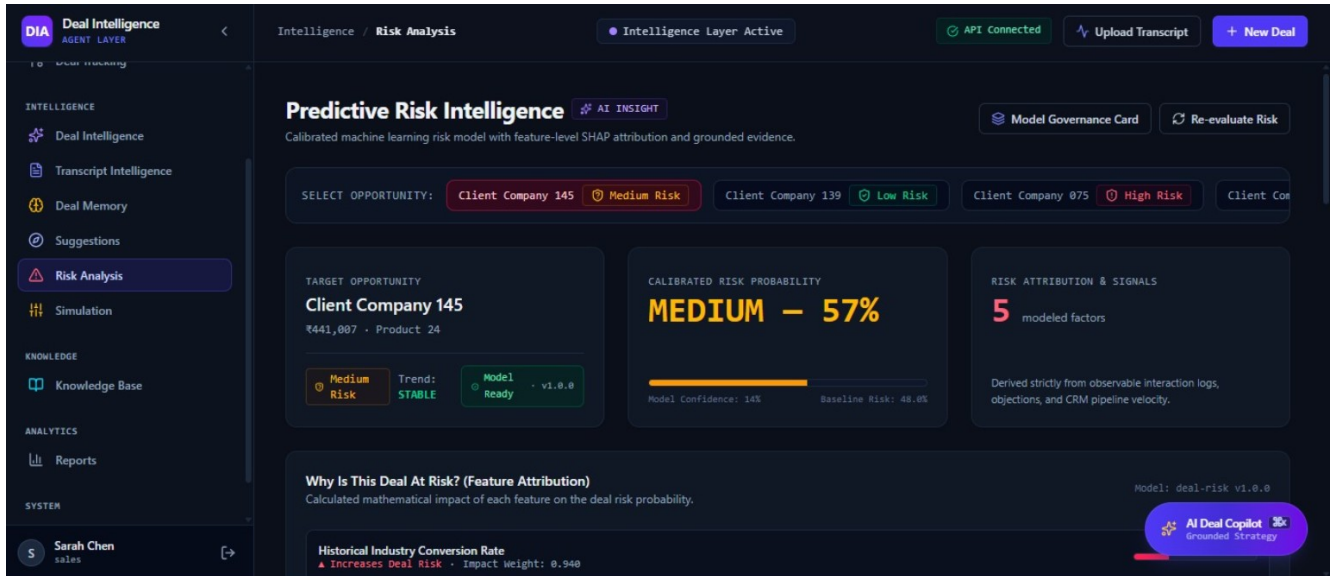


Figure 5. Predictive risk intelligence and feature-level risk signals.

A relevance floor is important because nearest-neighbor search always returns something. The system should be able to say: “No sufficiently similar deal found” when the evidence is weak.

5. Reflect: Turning Several Hits into One Argument

Recall gives the agent historical neighbors. Reflection compares what worked, what failed, and why, then produces a short synthesis that a sales representative can understand quickly.

Reflection output is treated as structured data rather than unrestricted prose. It includes a pattern, what worked, what failed, supporting deal IDs, and a confidence level. A suggestion without supporting deals should fail validation.

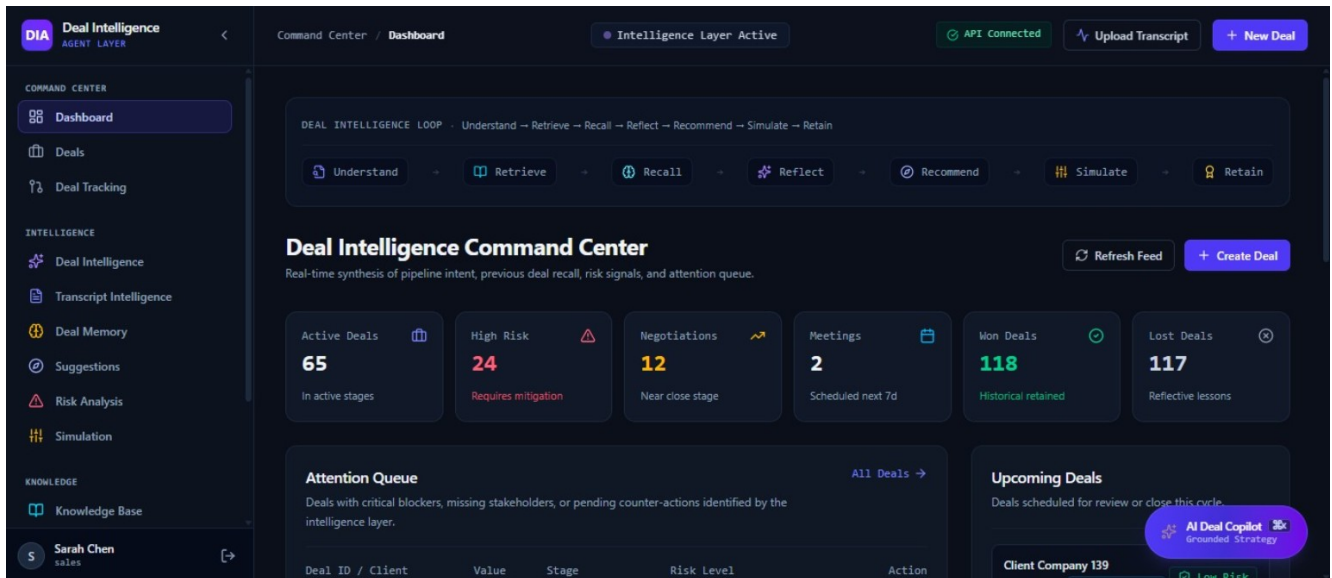


Figure 6. Command Center showing the Understand → Retrieve → Recall → Reflect → Recommend → Simulate → Retain loop.

6. Risk Analysis and Evidence

The interface can expose deal-risk signals alongside the historical-memory layer. The risk view shown in the project uses observable interaction logs, objections, CRM pipeline information, and modeled factors to explain why a deal is classified at a particular risk level.

The important design principle is grounding: risk signals and recommendations should be connected to evidence rather than presented as unsupported conclusions.

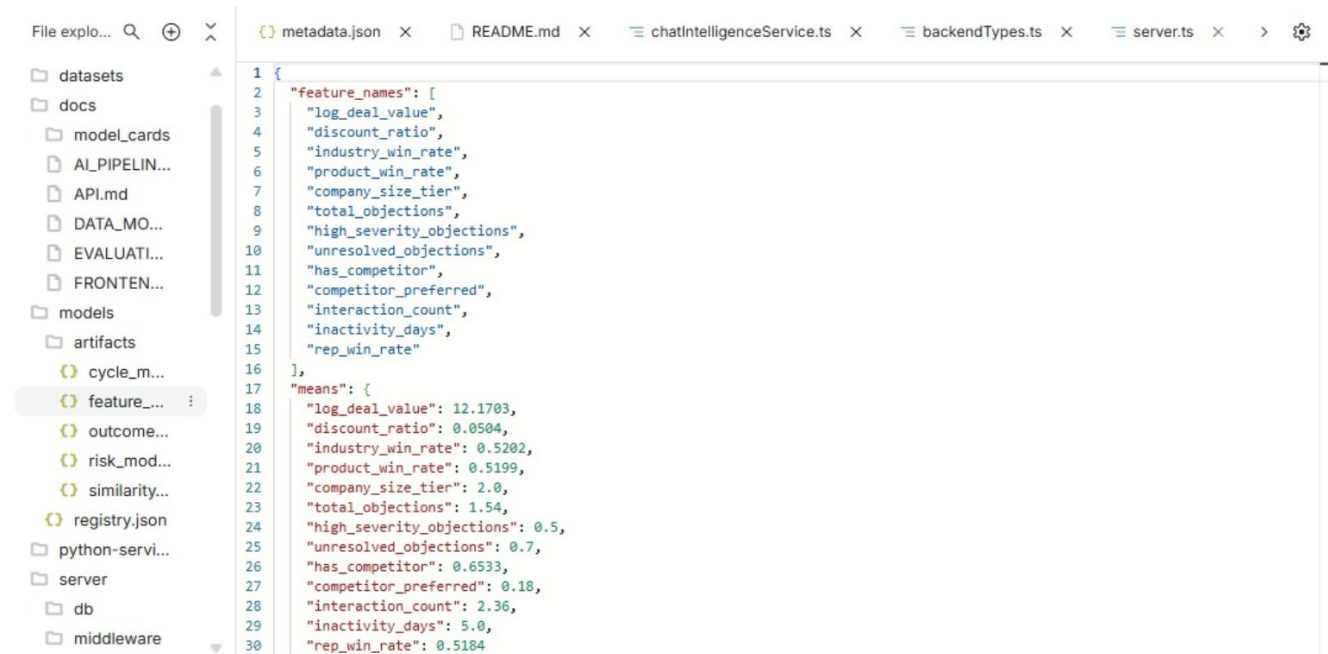


Figure 7. Risk analysis interface with calibrated risk probability and feature attribution.

7. What-If Simulation

The simulator builds on recalled historical patterns. A representative can change commercial levers such as discount, contract duration, and package, and compare scenarios against historical data.

The output is labelled as an estimate because it is derived from historical deals rather than a guaranteed forecast. This distinction prevents the interface from implying a level of precision that the available data cannot support.

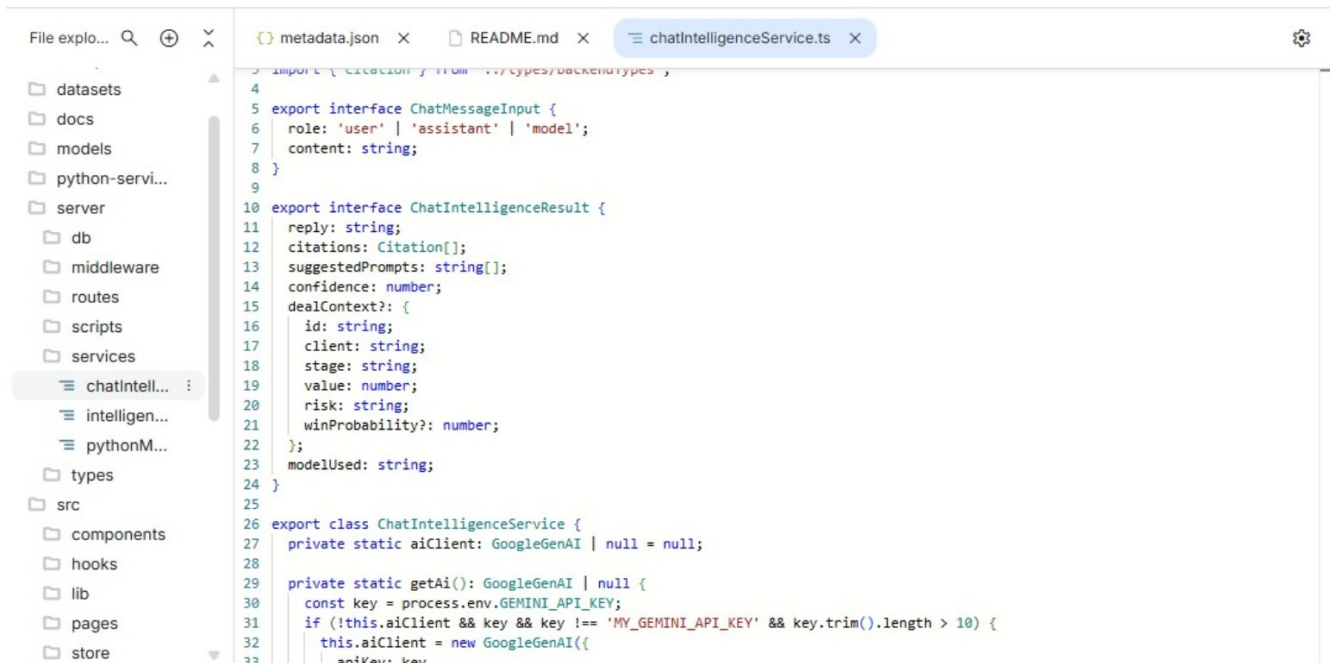


Figure 8. Strategic What-If Decision Lab for comparing commercial scenarios.

8. Lessons

1. Split knowledge from experience. Documents answer “what do we say?” Outcomes answer “what happened?” They need different storage and failure handling.

2. Store lessons, not logs. Agent memory is only as good as the distilled fact that is stored.

3. Put a floor under recall. Nearest-neighbor search always answers, so the “I don't know” path must be deliberate.

4. Make citations a schema constraint. A reflection should require supporting evidence.

5. Route memory through one door. A single memory service simplifies testing, tenant scoping, and telemetry.

The central idea is simple: a CRM records the deal, while a memory layer can retain lessons about how previous deals ended. That distinction turns retrieval into an experience-aware intelligence system.

Source basis: the supplied Deal Intelligence article and the project screenshots provided with this request.